Priority Scheduling

Choosing the runnable process with the highest priority

Definition

Priority scheduling is a scheduling method in which every process has a priority, and the runnable process with the highest priority is selected to run.

1. Why Priority Scheduling Is Needed

Main Idea

Round-robin scheduling treats runnable processes as equally important. Priority scheduling allows the system to represent that some processes should run before others.

Multiuser System

In a university system, the operating policy might give higher priority to administrative or faculty work than to ordinary student jobs.

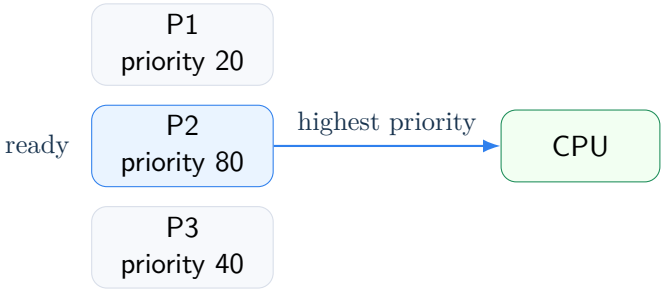
Single-User PC

Even on one user's computer, a real-time video display process may deserve higher priority than a background mail-sending daemon.

2. Basic Rule

Priority Rule

Among all runnable processes, run the process with the highest priority.



3. Avoiding Infinite Running

Problem

If a high-priority process is always runnable, lower-priority processes may never get the CPU.

Two Common Controls

- decrease the priority of the currently running process at each clock tick;
- assign each process a maximum time quantum, then run the next-highest-priority process when the quantum is used up.

Process Switch

If priority reduction makes the running process drop below another runnable process, the scheduler may switch to the higher-priority process.

4. Static Priorities

Static Priority

A **static priority** is assigned before or when the process starts and does not automatically change because of recent behavior.

Example	Possible Priority Policy
Military system	Processes started by generals may get priority 100, colonels 90, majors 80, and so on.
Commercial center	High-priority jobs may cost more per hour than medium- or low-priority jobs.
UNIX-like system	The `nice` command lets a user voluntarily lower a process's priority.

Policy-Based

Static priorities are useful when priority should reflect external policy, payment level, user category, or administrative importance.

5. Dynamic Priorities

Dynamic Priority

A **dynamic priority** is adjusted by the system while the process runs, often to improve responsiveness or system balance.

Why Dynamic Priorities Help

The scheduler can reward behavior that helps system performance, such as blocking quickly for I/O instead of using the whole CPU quantum.

6. Giving I/O-Bound Processes Good Service

I/O-Bound Motivation

An I/O-bound process often needs only a little CPU time before issuing its next I/O request. Running it quickly helps keep the I/O device busy while other processes compute.

Bad Delay

If an I/O-bound process waits too long for the CPU, it may occupy memory while doing nothing, and the I/O device may sit idle longer than necessary.

7. Dynamic Priority Formula

Priority Based on Quantum Use

One simple dynamic rule is:

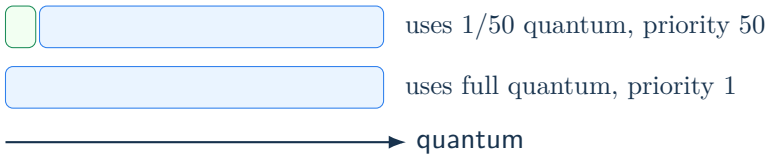
$$\text{priority} = \frac{1}{f}$$

where f is the fraction of the last quantum that the process used.

Last Quantum Use	Fraction f	Priority $1/f$
1 msec of 50 msec	1/50	50
25 msec of 50 msec	1/2	2
50 msec of 50 msec	1	1

Interpretation

A process that blocks quickly gets a high priority next time. A process that uses the whole quantum gets a low priority.



8. Priority Classes

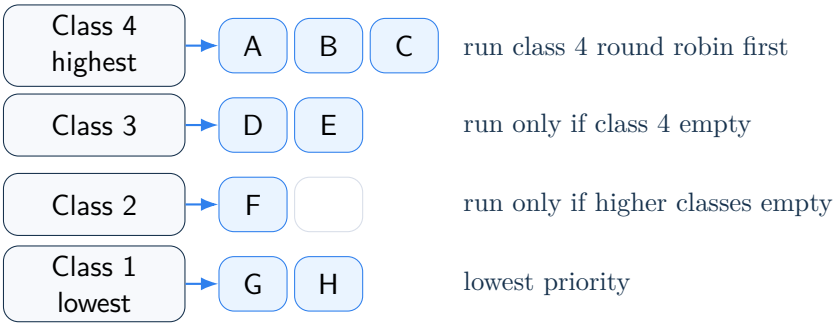
Priority Class

A **priority class** is a group of processes with the same broad priority level. The scheduler chooses among classes first, then chooses within a class.

Common Combination

Priority scheduling can be used among priority classes, while round-robin scheduling is used within each class.

9. Four Priority Classes



Scheduling Rule

As long as any class 4 process is runnable, run class 4 processes round robin. If class 4 is empty, run class 3 round robin. Continue downward only when all higher classes are empty.

10. Starvation

Starvation

Starvation occurs when a process waits indefinitely because higher-priority processes keep getting the CPU.

Priority Class Problem

If class 4 always has runnable processes, classes 3, 2, and 1 may never run.

Aging

One common fix is **aging**: gradually increase the priority of processes that have waited too long, so they eventually get a chance to run.

11. Priority Scheduling Sketch

```
while ready processes exist do
    choose the highest-priority nonempty class
    choose a process from that class

    run the process for one quantum

    if the process is still runnable then
        put it back in the appropriate ready queue
    end if

    adjust priorities if the policy requires it
end while
```

12. Priority Scheduling vs. Round Robin

Feature	Round Robin	Priority Scheduling
Basic assumption	All processes are roughly equal.	Some processes are more important.
Selection rule	Run the next process in queue order.	Run the highest-priority runnable process.
Fairness style	Fair by turn-taking.	Fair only within the chosen priority policy.
Risk	Poor service for important work.	Starvation of low-priority work.
Common mix	One ready queue.	Priority classes with round robin inside each class.

13. Big Picture

Main Conclusion

Priority scheduling lets the operating system prefer important processes over less important ones. Priorities may be static or dynamic, and priority classes are often combined with round-robin scheduling. The main danger is starvation of low-priority processes.

Exam Shortcut

Remember: priority scheduling runs the highest-priority runnable process; dynamic priorities can favor I/O-bound processes; priority classes often use round robin inside each class; starvation must be prevented by priority adjustment or aging.

14. Quick Review

Key Points

- Priority scheduling assigns a priority to each process.
- The highest-priority runnable process is selected to run.
- Priority scheduling is useful when not all processes are equally important.
- A high-priority process can be limited by lowering its priority over time or by using a maximum quantum.
- Static priorities are assigned from outside policy or user choice.
- Dynamic priorities are adjusted by the system during execution.
- I/O-bound processes can be rewarded because they often use only a small fraction of their quantum.
- A simple dynamic rule is priority $= 1/f$, where f is the fraction of the last quantum used.
- Priority classes can use priority scheduling among classes and round robin within a class.
- Lower-priority classes may starve if priorities are never adjusted.
- Aging raises the priority of long-waiting processes to reduce starvation.